



Python Project 5 - K-Means Clustering

In this project we want to explore a dataset which contains information on breakfast cereals. We are only going to consider the manufacturers, cereal ratings, sugar and carbohydrate levels. Some of the values in sugars and carbohydrate features have invalid data which we must address.

We want to answer the following questions.

1. What cereal manufacturers have the most/least varieties?
 2. What cereal has the highest rating?
 3. Investigate sugar levels in cereals
- Does each manufacturer provide cereals equally among the sugar levels?
 - How are the cereals clustered by sugar levels (low, medium, high)
 - What are the cereals which have the highest sugar level? lowest sugar levels?
 - How does your favorite breakfast cereal rank?

1. Investigate carbohydrate levels

- Cluster cereals in low, medium and high level of carbs
- What cereals are low sugar and low carbs?
- What cereals are high sugar and high carbs?

Steps:

- Input libraries and K-Means model from scikit-learn
- Create dataframe and print out a few lines
- Drop all features in dataframe except those we are using: Name, Manufacturer, Sugars and Carbohydrates, and Rating; alternately you can create a new dataframe with just the features you want
- Check to see if there are any NaN values in sugars or carbohydrates features
- Check to see if there are any invalid values (that is values <0) in sugars or carbohydrates features; if there are 2 or less cereals with these invalid values drop the cereal(s) from dataframe; otherwise replace negative values with 0.
- Plot number of cereals for each manufacturer; if any manufacturer has only 1 cereal, drop that data instance. Which manufacturer has the most varieties of cereals.
- In dataframe the manufacturer is listed by a single letter; for plotting purposes add a column giving the actual name and delete feature 'mfr'
- Use a plot to determine which manufacturer has cereals with the highest ratings.
- What cereal has the highest rating?
- Plot sugars levels vs manufacturer and determine which brand has lowest sugar levels.
- Cluster data using K-Means and the sugars feature with clusters low, middle and high sugar levels; print out cluster centroids; add cluster as feature in dataframe.
- Determine which cluster is associated with low, mid or high sugar levels; add feature giving sugar level corresponding to cluster
- Create a plot to show how cereals are distributed among sugar levels; what cereals have highest sugar levels? lowest sugar levels?
- Repeat the steps you did for investigating sugar levels using carbohydrate levels.
- Print out the cereals that are low sugars and low carbohydrates.
- Print out the cereals that are low sugars and high carbohydrates.



Data Set Description

This dataset is contained in the file `cereal.csv`

This dataset contains information on 77 different breakfast cereals. The features are

1. Cereal Name
2. Manufacturer
- A → American Home Food Products

G → General Mills

K → Kelloggs

N → Nabisco

P → Post

Q → Quaker Oats

R → Ralston Purina
1. Type (C → Cold, H → Hot)
2. Calories (per serving)
3. Protein (in grams)
4. Fat (in grams)
5. Sodium (in milligrams)
6. Fiber (in grams of dietary fiber)
7. Carbo (grams of complex carbohydrates)
8. Sugars (in grams)
9. Potass (milligrams of potassium)
10. Vitamins (possible values are 0, 25 or 100 indicating percent of FDA recommended vitamins and minerals)
11. Shelf (display shelf with possible values 1, 2, or 3 counting from floor)
12. Weight (weight of one serving in ounces)
13. Cups (number of cups in one serving)
14. Rating (Consumer ratings of cereal from 0 to 100)

```
# import libraries and K-Means function

import pandas as pd
from sklearn.cluster import KMeans
```

Create dataframe for data
df = pd.read_csv('cereal.csv')
df

	name object	mfr object	type object	calories int64	protein int64	fat int64	sodium int64	f
	100% Bran 1.3% 100% Natur... 1.3% 75 others 97.4%	K 29.9% G 28.6% 5 others 41.6%	C 96.1% H 3.9%	50 - 160	1 - 6	0 - 5	0 - 320	f
0	100% Bran	N	C	70	4	1	130	
1	100% Natural Bran	Q	C	120	3	5	15	
2	All-Bran	K	C	70	4	1	260	
3	All-Bran with Extr...	K	C	50	4	0	140	
4	Almond Delight	R	C	110	2	2	200	

77 rows, 16 cols 5 / page << < Page 1 of 16 > >> ↓

```
# drop features we aren't using or create a new dataframe with only the features we want
df2 = df[['name', 'mfr', 'carbo', 'sugars', 'rating']]
df2
```

	name object	mfr object	carbo float64	sugars int64	rating float64	
	100% Bran 1.3% 100% Natur... 1.3% 75 others 97.4%	K 29.9% G 28.6% 5 others 41.6%	-1.0 - 23.0 	-1 - 15 	18.042851 - 93.7... 	
0	100% Bran	N	5	6	68.402973	
1	100% Natural Bran	Q	8	8	33.983679	
2	All-Bran	K	7	5	59.425505	
3	All-Bran with Extr...	K	8	0	93.704912	
4	Almond Delight	R	14	8	34.384843	

77 rows, 5 cols 5 / page << < Page 1 of 16 > >> [↓](#)

```
# Check to see if any instances have NaN for entries using .info
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 77 entries, 0 to 76
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0    name    77 non-null      object
1    mfr      77 non-null      object
2    carbo    77 non-null      float64
3    sugars   77 non-null      int64
4    rating   77 non-null      float64
dtypes: float64(2), int64(1), object(2)
memory usage: 3.1+ KB
```

```
# Check to see if any cereals have negative values for sugars or carbohydrates; if 2 or less cereals, drop those
# data instances; otherwise replace negative values with 0
df2[df2['sugars']<0]
df2[df2['carbo']<0]
df2=df2[df2['sugars']>=0]
df2=df2[df2['carbo']>=0]
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 76 entries, 0 to 76
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0    name    76 non-null      object
1    mfr      76 non-null      object
2    carbo    76 non-null      float64
3    sugars   76 non-null      int64
4    rating   76 non-null      float64
dtypes: float64(2), int64(1), object(2)
memory usage: 3.6+ KB
```

```
# Address negative values
df2.loc[df2['sugars']<0, 'sugars']=0
df2.loc[df2['carbo']<0, 'carbo']=0
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 76 entries, 0 to 76
Data columns (total 5 columns):
#   Column   Non-Null Count  Dtype
---  ---
0    name    76 non-null    object
1    mfr      76 non-null    object
2    carbo    76 non-null    float64
3    sugars   76 non-null    int64
4    rating   76 non-null    float64
dtypes: float64(2), int64(1), object(2)
memory usage: 3.6+ KB
```

```
# set background grid for plots
# Update matplotlib
!pip install matplotlib==3.8.4

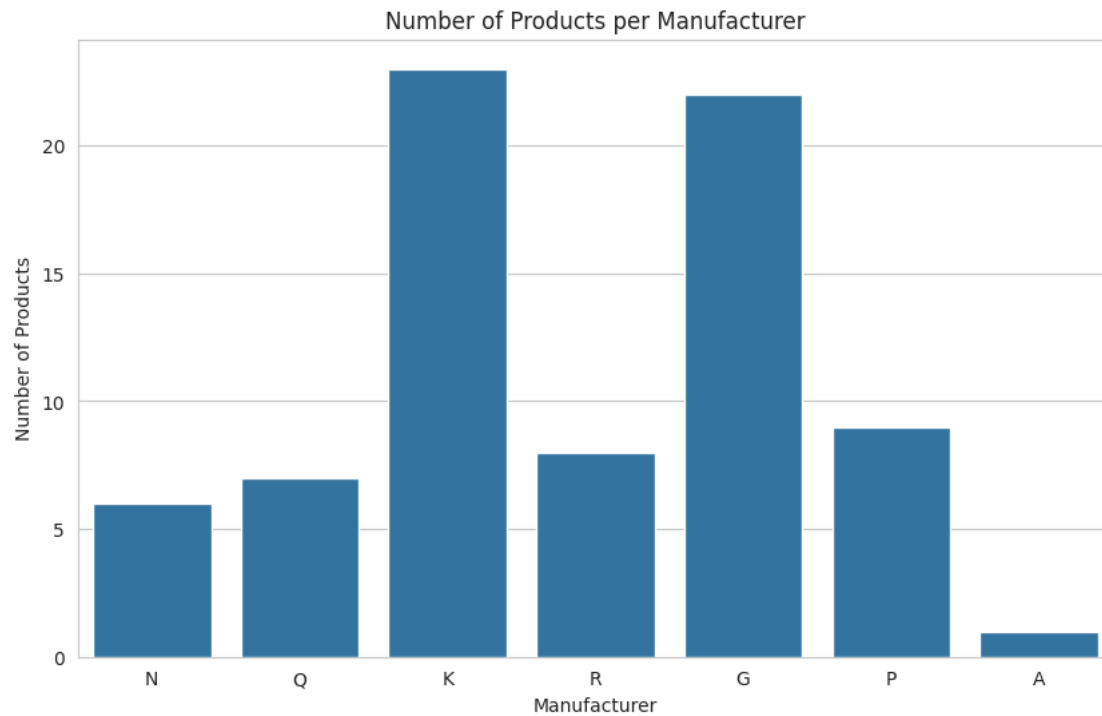
import seaborn as sns
import matplotlib.pyplot as plt

# Set the style for seaborn plots
sns.set_style("whitegrid")
```

```
Requirement already satisfied: matplotlib==3.8.4 in /root/venv/lib/python3.10/site-packages (3.8.4)
Requirement already satisfied: numpy>=1.21 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (1.25.2)
Requirement already satisfied: kiwisolver>=1.3.1 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (1.4.9)
Requirement already satisfied: contourpy>=1.0.1 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (1.3.2)
Requirement already satisfied: packaging>=20.0 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (25.0)
Requirement already satisfied: pillow>=8 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (3.2.5)
Requirement already satisfied: cycler>=0.10 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (0.12.1)
Requirement already satisfied: python-dateutil>=2.7 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (2.9.0.post0)
Requirement already satisfied: fonttools>=4.22.0 in /root/venv/lib/python3.10/site-packages (from matplotlib==3.8.4) (4.60.1)
Requirement already satisfied: six>=1.5 in /root/venv/lib/python3.10/site-packages (from python-dateutil>=2.7->matplotlib==3.8.4)
```

```
[notice] A new release of pip is available: 23.0.1 -> 25.3
[notice] To update, run: pip install --upgrade pip
```

```
# Plot number of products per manufacturer
plt.figure(figsize=(10, 6))
sns.countplot(data=df2, x='mfr')
plt.title('Number of Products per Manufacturer')
plt.xlabel('Manufacturer')
plt.ylabel('Number of Products')
plt.show()
```



```
# There is only one cereal from American Home Foods Company so we drop that data sample
#
# Type df[df.mfr == 'A'] and it will give you all data from this manufacturer.
# Make sure you set df = drop command; use command to make sure it was deleted

df2[df2['mfr'] == 'A']
df2 = df2[df2['mfr'] != 'A']

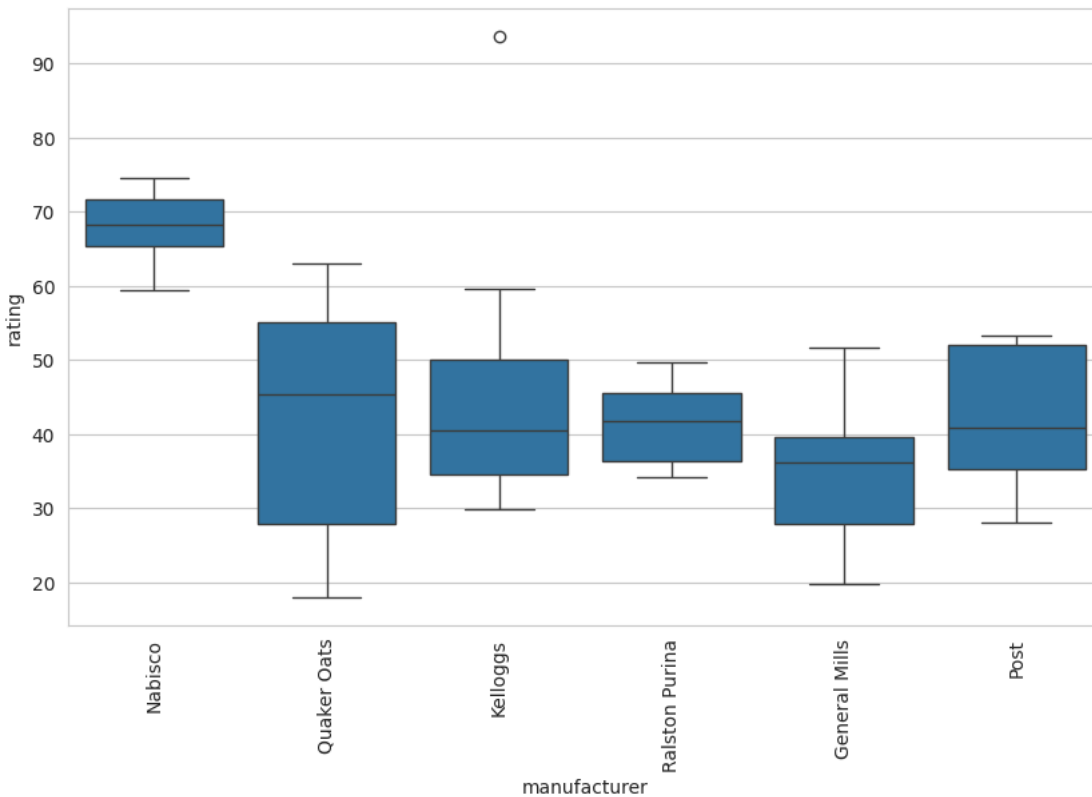
# check to make sure it was deleted
df2[df2['mfr'] == 'A']
```

name object	mfr object	carbo float64	sugars int64	rating float64	
No rows					
0 rows, 5 cols 10 / page					
<< < Page 0 of 0 > >> ↓					

```
# For plots we would like the name of manufacturer instead of just "N" or "Q"
# Use .map or .apply
#
# Then drop 'mfr' column
#
mfr_ext = {'G': 'General Mills', 'K': 'Kelloggs', 'N': 'Nabisco', 'P': 'Post', 'Q': 'Quaker Oats', 'R': 'Ralston Purina'}
df2['manufacturer'] = df2['mfr'].map(mfr_ext)
df2 = df2.drop(['mfr'], axis=1)
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 75 entries, 0 to 76
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    name        75 non-null    object
1   carbo        75 non-null    float64
2   sugars       75 non-null    int64
3   rating       75 non-null    float64
4   manufacturer 75 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 3.5+ KB
```

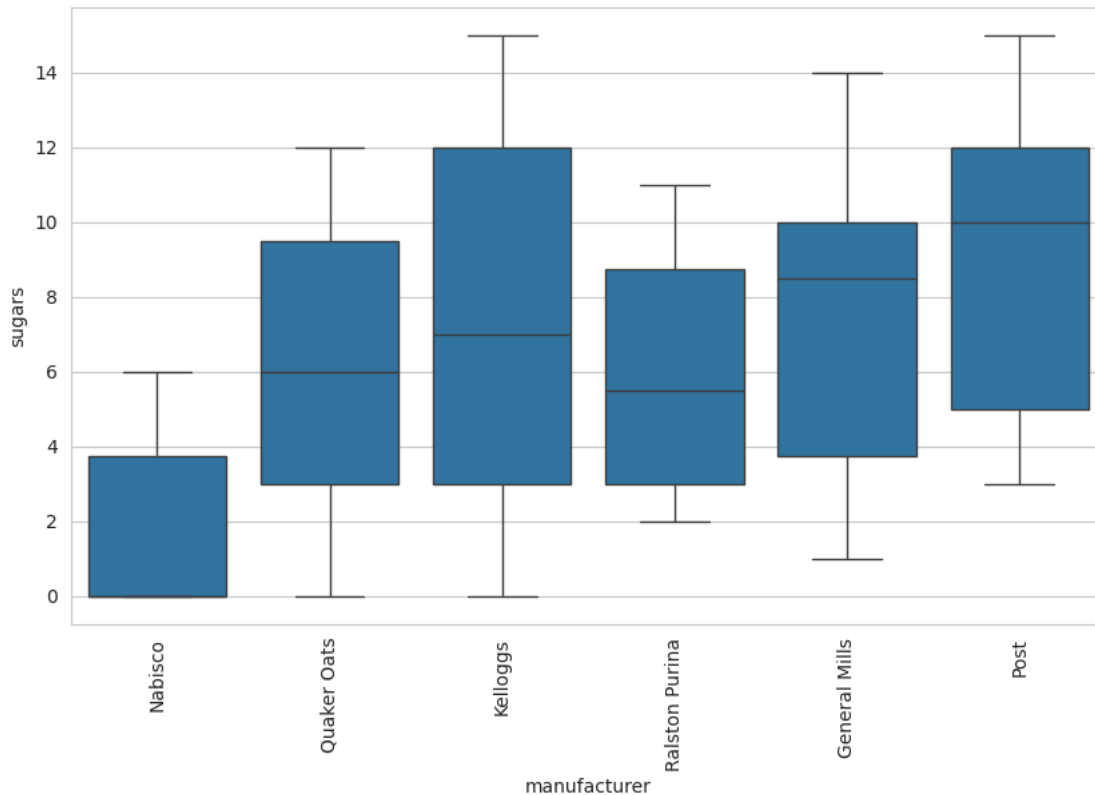
```
# Plot to see which manufacturer has highest rated cereals
plt.figure(figsize=(10, 6))
sns.boxplot(x='manufacturer', y='rating', data=df2)
plt.xticks(rotation=90)
plt.show()
```



```
# Which cereal is rated highest?
# Kelloggs is the manufacturer with the highest rated cereal
df2[df2['rating'] == df2['rating'].max()].manufacturer
```

```
3    Kelloggs
Name: manufacturer, dtype: object
```

```
# Look at sugars per brand by plotting
plt.figure(figsize=(10, 6))
sns.boxplot(x='manufacturer', y='sugars', data=df2)
plt.xticks(rotation=90)
plt.show()
```



Cluster by sugars into highest, middle and lowest levels; random initial guess

```
# Get data for clustering
import numpy as np
from sklearn.cluster import KMeans

X_sugars = df2[['sugars']].values
```

```
# Form model, fit data and print out cluster centers
kmeans = KMeans(n_clusters=3, random_state=123)
kmeans.fit(X_sugars)
print("Cluster centers (sugar levels):", kmeans.cluster_centers_.flatten())
```

```
Cluster centers (sugar levels): [ 6.84      1.95833333 12.03846154]
```

```
# Add column to dataframe for this clusters, say sugars_clusters
```

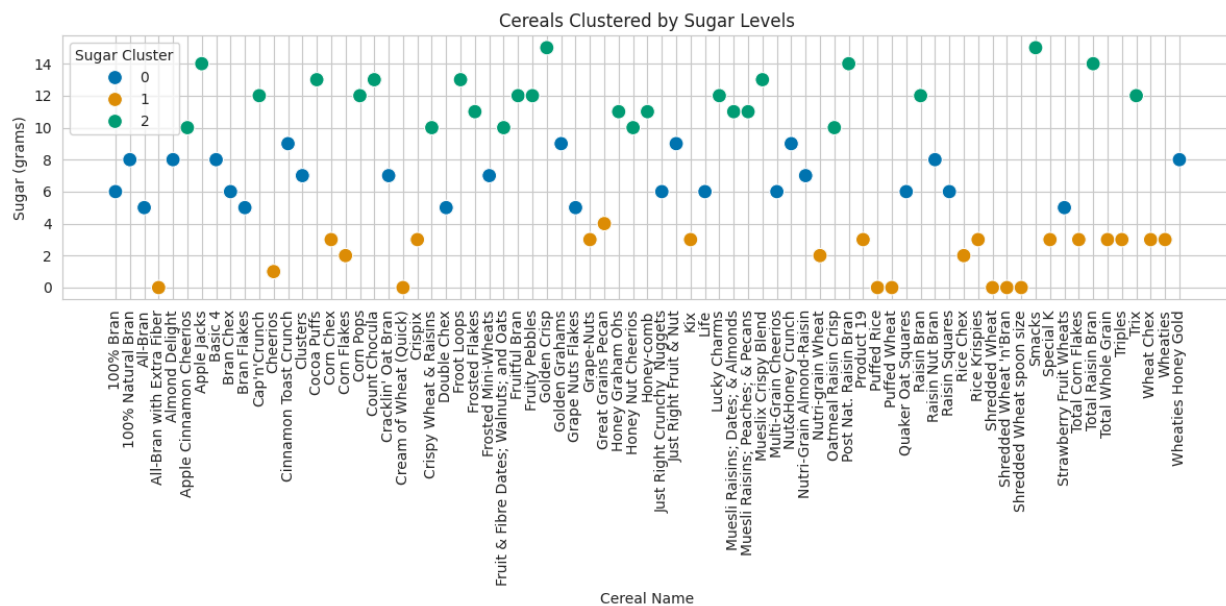
```
df2['sugars_cluster'] = kmeans.labels_  
df2.head()
```

	name object	carbo float64	sugars int64	rating float64	manufacturer obj...	sugars_cluster in...	
0	100% Bran	5	6	68.402973	Nabisco	0	
1	100% Natural Bran	8	8	33.983679	Quaker Oats	0	
2	All-Bran	7	5	59.425505	Kelloggs	0	
3	All-Bran with Extr...	8	0	93.704912	Kelloggs	1	
4	Almond Delight	14	8	34.384843	Ralston Purina	0	

5 rows, 6 cols 10 / page << < Page 1 of 1 > >> [↓](#)

```
# Plot clusters
```

```
plt.figure(figsize=(12,6))  
sns.scatterplot(data=df2, x='name', y='sugars', hue='sugars_cluster', palette='colorblind', s=100)  
plt.xticks(rotation=90)  
plt.xlabel('Cereal Name')  
plt.ylabel('Sugar (grams)')  
plt.title('Cereals Clustered by Sugar Levels')  
plt.legend(title='Sugar Cluster')  
plt.tight_layout()  
plt.show()
```



```
# Determine which cluster number corresponds to lowest, middle and highest level and create a new
# column in dataframe using .map
#
# First, get the cluster centroids and sort them
centers = kmeans.cluster_centers_.flatten()
sorted_indices = centers.argsort()

#creating mappiong from levels to cluster numbers
cluster_mapping = {sorted_indices[0]: 'Low', sorted_indices[1]: 'Medium', sorted_indices[2]: 'High'}

# new column
df2['sugar_level'] = df2['sugars_cluster'].map(cluster_mapping)
df2[['name', 'sugars', 'sugars_cluster', 'sugar_level']].head()
```

	name object	sugars int64	sugars_cluster in...	sugar_level object	
0	100% Bran	6	0	Medium	
1	100% Natural Bran	8	0	Medium	
2	All-Bran	5	0	Medium	
3	All-Bran with Extr...	0	1	Low	
4	Almond Delight	8	0	Medium	

5 rows, 4 cols / page << < Page of 1 > >> [↓](#)

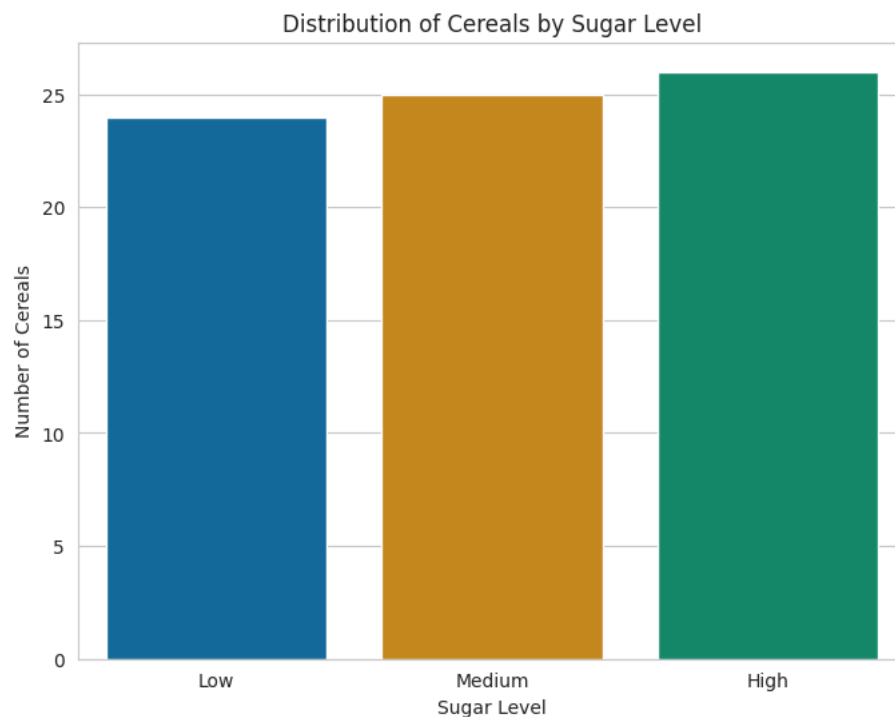
```
# How are cereals distributed among the 3 levels?
plt.figure(figsize=(8,6))
sns.countplot(data=df2, x='sugar_level', palette='colorblind', order=['Low', 'Medium', 'High'])

plt.xlabel('Sugar Level')
plt.ylabel('Number of Cereals')
plt.title('Distribution of Cereals by Sugar Level')
plt.show()
```

/tmp/ipykernel_99/4158846650.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.countplot(data=df2, x='sugar_level', palette='colorblind', order=['Low', 'Medium', 'High'])
```



```
# Which cereals have the highest sugar levels
highest_sugar = df2[df2['sugar_level'] == 'High']
highest_sugar = highest_sugar.sort_values(by='sugars', ascending=False)
highest_sugar[['name', 'sugars', 'sugar_level']].head(10)
```

	name object	sugars int64	sugar_level object	
	Smacks 10% Golden Crisp ... 10% 8 others 80%	12 - 15 	High 100%	
66	Smacks	15	High	
30	Golden Crisp	15	High	
52	Post Nat. Raisin B...	14	High	
70	Total Raisin Bran	14	High	
6	Apple Jacks	14	High	

10 rows, 3 cols / page << < Page of 1 > >> [↓](#)

```
# Which cereals have the lowest sugar levels
lowest_sugar = df2[df2['sugar_level'] == 'Low']
lowest_sugar = lowest_sugar.sort_values(by='sugars', ascending=True)
lowest_sugar[['name', 'sugars', 'sugar_level']].head(10)
```

	name object	sugars int64	sugar_level object	
	All-Brn wit... . 10% Shredded 10% 8 others 80%	0 - 2 	Low 100%	
3	All-Brn with Extr...	0	Low	
65	Shredded Wheat ...	0	Low	
64	Shredded Wheat...	0	Low	
63	Shredded Wheat	0	Low	
55	Puffed Wheat	0	Low	

10 rows, 3 cols / page << < Page of 1 > >> [↓](#)

```
# If you eat a particular cereal like Apple Jacks, Froot Loops, etc. what cluster is it in?
my_cereal = 'Apple Jacks'

cluster = df2.loc[df2['name'] == my_cereal, 'sugars_cluster'].values[0]
level = df2.loc[df2['name'] == my_cereal, 'sugar_level'].values[0]

print(f"The data instance and sugar cluster for {my_cereal} is {cluster} and the level is {level}")
```

The data instance and sugar cluster for Apple Jacks is 2 and the level is High

Repeat calculations and plots for carbohydrates instead of sugars

```
x_carbo = df2[['carbo']]
```

```
# Form model, fit data and print out cluster centers
kmeans_carbo = KMeans(n_clusters=3, random_state=123)
kmeans_carbo.fit(x_carbo)
print("Carbohydrate cluster centers (grams):")
print(kmeans_carbo.cluster_centers_)
```

```
Carbohydrate cluster centers (grams):
[[14.81944444]
 [10.38636364]
 [20.41176471]]
```

```
# adding cluster to df
df2['carbo_cluster'] = kmeans_carbo.labels_
df2.head()
```

	name object	carbo float64	sugars int64	rating float64	manufacturer obj...	sugars_cluster in...	sugar_level object	c
0	100% Bran	5	6	68.402973	Nabisco	0	Medium	
1	100% Natural Bran	8	8	33.983679	Quaker Oats	0	Medium	
2	All-Bran	7	5	59.425505	Kelloggs	0	Medium	
3	All-Bran with Extr...	8	0	93.704912	Kelloggs	1	Low	
4	Almond Delight	14	8	34.384843	Ralston Purina	0	Medium	

5 rows, 8 cols / page << < Page of 1 > >> [↓](#)

What cereals are high carbs and low sugar? What are low carbs and low sugar?

```
high_carbo_low_sugar = df2[(df2['carbo_level'] == 'High') & (df2['sugar_level'] == 'Low')]
print("Cereals with high carbohydrates and low sugar:")
print(high_carbo_low_sugar[['name', 'carbo', 'sugars', 'carbo_level', 'sugar_level']])

low_carbo_low_sugar = df2[(df2['carbo_level'] == 'Low') & (df2['sugar_level'] == 'Low')]
print("Cereals with low carbohydrates and low sugar:")
print(low_carbo_low_sugar[['name', 'carbo', 'sugars', 'carbo_level', 'sugar_level']])
```

Cereals with high carbohydrates and low sugar:

	name	carbo	sugars	carbo_level	sugar_level
15	Corn Chex	22.0	3	High	Low
16	Corn Flakes	21.0	2	High	Low
20	Cream of Wheat (Quick)	21.0	0	High	Low
21	Crispix	21.0	3	High	Low
40	Kix	21.0	3	High	Low
50	Nutri-grain Wheat	18.0	2	High	Low
53	Product 19	20.0	3	High	Low
61	Rice Chex	23.0	2	High	Low
62	Rice Krispies	22.0	3	High	Low
64	Shredded Wheat 'n'Bran	19.0	0	High	Low
65	Shredded Wheat spoon size	20.0	0	High	Low
69	Total Corn Flakes	21.0	3	High	Low
72	Triples	21.0	3	High	Low

Cereals with low carbohydrates and low sugar:

	name	carbo	sugars	carbo_level	sugar_level
3	All-Bran with Extra Fiber	8.0	0	Low	Low
55	Puffed Wheat	10.0	0	Low	Low

```
# Get cluster centers and sort them to determine which is Low, Medium, High
centers_carbo = kmeans_carbo.cluster_centers_.flatten()
cluster_order_carbo = centers_carbo.argsort()

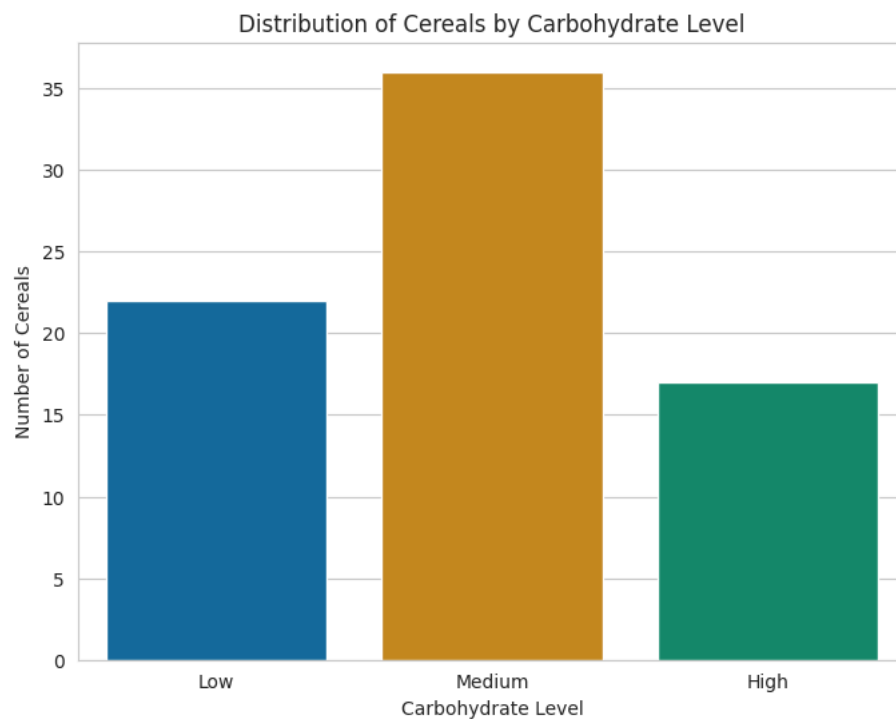
cluster_labels_carbo = ['Low', 'Medium', 'High']
cluster_mapping_carbo = {cluster_order_carbo[i]: cluster_labels_carbo[i] for i in range(3)}

df2['carbo_level'] = df2['carbo_cluster'].map(cluster_mapping_carbo)
df2.head()
```

	name object	carbo float64	sugars int64	rating float64	manufacturer obj...	sugars_cluster in...	sugar_level object	c
0	100% Bran	5	6	68.402973	Nabisco	0	Medium	
1	100% Natural Bran	8	8	33.983679	Quaker Oats	0	Medium	
2	All-Bran	7	5	59.425505	Kelloggs	0	Medium	
3	All-Bran with Extr...	8	0	93.704912	Kelloggs	1	Low	
4	Almond Delight	14	8	34.384843	Ralston Purina	0	Medium	

5 rows, 9 cols / page << < Page of 1 > >> [↓](#)

```
plt.figure(figsize=(8,6))
sns.countplot(data=df2, x='carbo_level', hue='carbo_level', order=['Low', 'Medium', 'High'], palette='colorblind')
plt.xlabel('Carbohydrate Level')
plt.ylabel('Number of Cereals')
plt.title('Distribution of Cereals by Carbohydrate Level')
plt.show()
```



```
#highest
highest_carbo = df2[df2['carbo'] == df2['carbo'].max()]
print("Cereals with highest carbohydrate content:")
print(highest_carbo[['name', 'carbo', 'carbo_level']])

#lowest
lowest_carbo = df2[df2['carbo'] == df2['carbo'].min()]
print("\nCereals with lowest carbohydrate content:")
print(lowest_carbo[['name', 'carbo', 'carbo_level']])
```

Cereals with highest carbohydrate content:

	name	carbo	carbo_level
61	Rice Chex	23.0	High

Cereals with lowest carbohydrate content:

	name	carbo	carbo_level
0	100% Bran	5.0	Low